

Connect Four Mini AI – Minimax

Project Report

1. Problem Formulation

The project implements a two-player Connect Four game on a 6×7 board. The objective is to create an AI player that selects moves using a depth-limited Minimax search with a static evaluation function.

States: A state is the current arrangement of X, O, and empty positions on the 6×7 board. **Actions:** A legal action is choosing an available column from 1 to 7 and dropping a piece into it. **Goal test:** The game reaches a goal state when either player connects four pieces horizontally, vertically, or diagonally. A full board without a winner is a draw. **Cost:** The practical search cost is the number of game-tree nodes explored by Minimax for an AI decision.

Players are represented as X for the human and O for the AI. The AI is the maximizing player and the human is treated as the minimizing player.

2. Approach

The selected algorithm is Depth-Limited Minimax. For each legal AI move, the program simulates the move and recursively explores possible future AI and human moves. The search depth is fixed at 4, which limits computation while still allowing the AI to look several moves ahead.

Terminal states are given large values: an AI win receives +100000 and a human win receives -100000. If the depth limit is reached without a winner, the static evaluation function is used.

The evaluation function gives +2 for each AI piece and -2 for each human piece. Each AI piece in the center column receives an additional +3. This gives the AI a simple preference for occupying the center while considering the overall number of pieces.

Board condition	Score
AI piece (O)	+2
Human piece (X)	-2
AI piece in center column	Additional +3
AI win	+100000
Human win	-100000

3. Complexity

For Connect Four, the branching factor is at most 7 because a player can choose one of seven columns. With a depth limit d , a straightforward Minimax search has approximately $O(b^d)$ time complexity, where b is the branching factor. For this implementation, $b \leq 7$ and $d = 4$, giving a practical upper-bound style estimate of $O(7^4)$ leaf-level growth before accounting for early terminal states and changing numbers of legal moves.

The implementation creates a copied board for each simulated move. Its recursive search therefore also requires memory for the current recursion path and temporary board copies. The practical limitation is that increasing the search depth causes the number of explored nodes to grow rapidly, which increases response time.

4. Results

The implementation was executed successfully in a GitHub Codespace. The game displayed the board, accepted a human move, performed an AI search, and returned a valid AI move.

Observed game output:

```
AI thinking...
AI chose 4 | Nodes: 2800
```

The observed run demonstrates that the AI performed a Minimax search and reported the number of explored nodes for that board position. The node count is position-dependent and can change from one turn to another.

Automated test result:

3 tests collected → 3 passed in 0.73s

The tests checked board dimensions, win detection, and whether the AI selected a valid move. These tests passed successfully in the project environment.

5. Reflection

The main challenge was implementing the recursive Minimax search while correctly handling board copies, alternating maximizing and minimizing turns, terminal states, and the depth limit. Another practical challenge was verifying that the AI returned legal moves and that the test environment could import the project correctly.

A future improvement would be a stronger heuristic that evaluates groups of two, three, and four connected pieces for both players. Alpha-Beta pruning could also be added to reduce the number of nodes searched and allow a greater search depth.

AI/source assistance note: AI tools were used to help understand Minimax concepts, organize documentation, and troubleshoot implementation issues; the final project was executed and tested by the student.

Conclusion: The project successfully demonstrates a Connect Four AI using depth-limited Minimax and a static evaluation function. The implementation was executed successfully and its automated tests passed.